

IIITA Timetabling System

*A thesis submitted in partial fulfillment of the requirements
for the award of the degree of*

BACHELOR OF TECHNOLOGY



By:-

ESHANT

RAHUL ROY

LAKAVATH PEER SINGH

Enrollment No.

IIT2023198

IIT2023196

IIT2023197

Under the Supervision of

DR. GAURAV SRIVASTAVA

to the

DEPARTMENT OF INFORMATION TECHNOLOGY

**INDIAN INSTITUTE OF INFORMATION TECHNOLOGY,
ALLAHABAD**

Wednesday 7th May, 2026



भारतीय सूचना प्रौद्योगिकी संस्थान इलाहाबाद
Indian Institute of Information Technology Allahabad

An Institute of National Importance by Act of Parliament

Deoghat Jhalwa, Prayagraj 211015 (U.P.) India

Ph: 0532-2922025, 2922067; Web: www.iiita.ac.in; Email: contact@iiita.ac.in

CERTIFICATE

It is certified that the work contained in the thesis titled “IIITA Timetabling System” by Eshant, Rahul Roy, Lakavath Peer Singh has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

Dr. Gaurav Srivastava
Department of Information Technology
IIIT Allahabad



भारतीय सूचना प्रौद्योगिकी संस्थान इलाहाबाद
Indian Institute of Information Technology Allahabad

An Institute of National Importance by Act of Parliament

Deoghat Jhalwa, Prayagraj 211015 (U.P.) India

Ph: 0532-2922025, 2922067; Web: www.iiita.ac.in; Email: contact@iiita.ac.in

CANDIDATE DECLARATION

We, Eshant (IIT2023198), Rahul Roy (IIT2023196), and Lakavath Peer Singh (IIT2023197), hereby certify that the thesis titled “IIITA Timetabling System” has been submitted by our in partial fulfillment of the requirements for the Degree of Bachelor of Technology in the Department of Information Technology at the Indian Institute of Information Technology, Allahabad.

We acknowledge that plagiarism includes:

1. Reproducing another person's work (fully or partially) or ideas and presenting them as our own.
2. Copying or paraphrasing another person's work without appropriate attribution.
3. Engaging in literary theft by reproducing a unique literary construct without crediting its source.

We affirm that due credit has been given to all original authors and sources through proper citations for words, ideas, diagrams, graphics, computer programs, experiments, results, and websites that are not our own contributions. Direct quotations have been appropriately marked, and sources have been properly referenced. Furthermore, we declare that no portion of our work is plagiarized. In the event of a plagiarism claim, we accept full responsibility for the contents of this thesis. We acknowledge that our supervisor may not be able to verify the originality of every aspect of our work.

Place: **IIITA**

Date: **07/05/2026**

Eshant (IIT2023198)

Rahul Roy (IIT2023196)

Lakavath Peer Singh (IIT2023197)

IIITA Timetabling System

ABSTRACT

University timetabling is a complex problem, or rather, it is an NP-hard problem. Even small campuses have hundreds of constraints to schedule for: courses, professors, rooms, and groups of students. At IIIT Allahabad, the timetables were generated manually in spreadsheets, which was time-consuming, error-prone, and not scalable. In this thesis, we created the "IIITA Timetabling System", a web-based solution to automate and digitise the entire timetable scheduling pipeline.

We implemented a heuristic-based Excel parser that can process raw, unstructured timetable Excel spreadsheets, extracting information such as subject codes, faculty names and room numbers, and loads them into a cloud-based PostgreSQL database using Supabase.

The main contribution is an automated timetable scheduler based on a (1+1) Evolution Strategy (ES). We represent a timetable as a chromosome with Gene decisions (day, time-slot, room) and evaluate candidate solutions using a fitness function with 10 hard constraints (room / professor / group clashes, break boundary limits, labs, elective synchronisation, etc.) and 3 soft constraints (minimising gaps for students, maximising room occupancy and keeping professors in one side of the day). The algorithm adjusts step sizes using Rechenberg's classic 1/5th rule, restarts when trapped in local optimums and can be initialised with a given timetable. To prevent clashes across semesters, we populate already scheduled sessions from published timetables.

Since no solver can anticipate every conceivable real-world preference, we also provide a drag-and-drop Timetable Editor. Administrators can visually rearrange the sessions, with conflicts displayed in real-time, and LNS (Large Neighbourhood Search) auto-fix with one click. A Master View displays slots from all published timetables so conflicts across semesters are instantly apparent.

In daily life, however, the system allows customized views for each stakeholder, from the students' view of their individual schedule to professors' schedules and a master occupancy map for the entire university campus, plus the ability to find available rooms using our Free Room Finder module. In addition, we developed an extensive Intelligent Generator to guide administrators step-by-step.

React 19, TypeScript, and Tailwind CSS form the technology stack used throughout the application, making it very flexible and modern in design. The use of the Google Sheets API makes data export elegant, while PDF creation is possible locally as well. In the context of this thesis, topics such as the theoretical framework of the timetabling problem, the technology stack and architecture, evolutionary algorithm with constraints, and interactive editor components are discussed.

Contents

Abstract	iii
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Objectives of the Study	3
1.4 Scope and Limitations	5
1.5 Organization of the Thesis	5
2 Literature Review	7
2.1 The University Timetabling Problem (UTP)	7
2.1.1 Constraint Satisfaction and Complexity	7
2.2 Algorithmic Approaches to Timetabling	9
2.2.1 Exact Methods	9
2.2.2 Heuristic and Meta-Heuristic Algorithms	9
2.3 Evolution of Timetable Management Systems	10
2.3.1 Legacy Systems: Spreadsheets and Desktop Applications	10
2.3.2 Modern Web-Based Systems	11
2.4 Identification of Gaps in Existing Solutions	12
3 Technologies and Frameworks	14
3.1 Frontend Architecture: React 19 and TypeScript	14
3.1.1 Performance and Data Reliability	15
3.2 Build and Styling: Vite and Tailwind CSS	15
3.3 Backend and Data Ingestion: Supabase	16
3.4 Solver Engine: Pure TypeScript Architecture	16
3.5 External Integrations: Reporting and Export	17
4 System Architecture and Design	18
4.1 Application Topology	18
4.2 Database Schema Design	19
4.2.1 Core Entities	19
4.2.2 The Central Hub: timetable_slots	20
4.2.3 Solver Data Structures	21
4.3 Asynchronous Processing Design	22
5 Implementation and Results	25
5.1 Introduction to the Optimization Engine	25
5.2 Mathematical Representation of the Schedule	25

5.3	Mathematical Formulation of Constraints	27
5.3.1	Hard Constraints (The Feasibility Space)	27
5.3.2	Soft Constraints (The Quality Space)	30
5.4	The (1+1) Evolution Strategy Solver	30
5.4.1	Strict Elitism	30
5.4.2	Mutation Operators	31
5.4.3	Adaptive Variance (The Brain)	31
5.4.4	Stagnation Recovery	32
5.5	Interactive Editor & Large Neighbourhood Search (LNS)	32
5.6	Results and Application Features	35
5.6.1	Algorithmic Convergence	35
5.6.2	The Interactive Drag-and-Drop Grid	35
5.6.3	LNS Auto-Fix Execution	35
5.6.4	Export and PDF Generation	36
5.7	Dynamic Time Grid Generation	36
5.8	Multi-Dimensional Clash Detection Algorithm	37
5.9	Inverted Logic: The Free Room Finder	38
5.10	React Component Hierarchy	40
5.11	Role-Based Viewer Implementations	40
5.11.1	Student Cohort View	40
5.11.2	Master Occupancy Viewer	41
5.12	Export Mechanisms and Results	41
5.12.1	Client-Side PDF Generation	41
5.12.2	Google Sheets API Export	42
5.12.3	User Interface Evaluation	42
5.13	The Interactive Timetable Editor	42
5.13.1	Drag-and-Drop Slot Rearrangement	43
5.13.2	Inline Editing	43
5.13.3	Real-Time Conflict Visualization	43
5.13.4	Feasibility Check	44
5.13.5	LNS Auto-Fix	44
6	Conclusion and Future Work	46
6.1	Summary of Contributions	46
6.2	Current Limitations and Proposed Enhancements	48
6.3	Concluding Remarks	48
A	Core Algorithm Snippets	51
A.1	Dynamic Grid Generation Logic	51
A.2	Real-Time Clash Detection Filter	51
A.3	Solver Fitness Evaluation Function	52
A.4	(1+1)-ES Solver Main Loop	53
A.5	LNS Auto-Fix for the Editor	54

List of Figures

3.1	High-Level System Architecture and Component Interaction	15
4.1	Entity-Relationship Diagram of the Complete Database Schema	19
4.2	UML Class Diagram of the Solver Data Model	21
4.3	Data Ingestion Pipeline for Excel Timetables	23
5.1	Sequence Diagram: End-to-End Intelligent Generator Workflow	26
5.2	Activity Diagram: (1+1)-ES Solver Execution Flow	33
5.3	Activity Diagram: LNS Destroy-and-Repair Auto-Fix	34
5.4	Inverted Logic Flowchart for the Free Room Finder Algorithm	39
5.5	Hierarchical Structure of the Core React Components	40
5.6	Activity Diagram: LNS Destroy-and-Repair Auto-Fix	45

List of Tables

2.1	Comparison of Timetabling Methodologies	13
3.1	Summary of the Technology Stack	14
4.1	Schema Specification for <code>timetable_slots</code>	20
5.1	Hard Constraints Enforced by the Solver	29
5.2	Types of Scheduling Clashes Detected	37
5.3	Role-Based Component Mapping and Data Scope	40
6.1	System Limitations and Future Enhancements	49

Chapter 1

Introduction

Each academic semester, the university faces the same challenging problem. How do you allocate hundreds of classes, a number of lecturers, and a fixed set of class rooms to a semester-wide timetable? This problem is known as the University Timetabling Problem (UTP) and it has been proven NP-hard, i.e., that there is no efficient algorithm for solving the problem optimally. This chapter will give a brief overview of the IITA Timetabling System, describe its rationale, and outline the remaining chapters in this thesis.

1.1 Background and Motivation

An academic timetable involves assigning lectures, tutorials, and laboratory classes to a particular time slot, venue, and lecturer, while taking into account the limitations imposed by the situation. There are some constraints that are rigid, in that it is not possible to double book the lecturer or run two classes in one room at the same time. On the other hand, there are certain soft constraints which include having the minimum amount of gap between lectures, and the professors prefer lecturing in one half of the day only.

IIT Allahabad is one among the many colleges in India where the scheduling process remains manual. Timetables are created manually using Microsoft Excel, and conflicts are manually resolved by the academic coordinators. Even though Microsoft Excel is highly

customizable and flexible, it does not have any sense of the meaning of a “professor” or “room”. It is up to the manual efforts of the people preparing the timetables to make sure that such problems are avoided.

The issues do not stop even after the timetable is released. Using a static PDF or Excel file will result in the need to send out the whole document every time there is a change, such as a classroom change or an instructor change. The students will have to work with outdated versions, and no one will be able to answer basic queries such as "What classrooms are available at 2 PM on Wednesday?"

These annoyances were what pushed us to develop the IIITA Timetabling System. We sought a unified solution that would become the ultimate authority on all scheduling information. This system should be capable of automatically generating conflict-free timetables, allowing the administrators to adjust them interactively, and providing individualized timetables for each stakeholder.

1.2 Problem Statement

The underlying issue we sought to address is simple: The manual creation and management of timetables via spreadsheets at IIITA is inefficient, error-prone, and inherently inflexible. Specifically, the existing process suffers from:

- **Unstructured Data:** Our schedule is created in excel sheets with merged cells, non-matching names, and relations that are beyond any computer program’s comprehension.
- **Manual Conflict Resolution:** To check whether there are problems like clash of classrooms, unavailability of certain teachers or courses, one needs to do a whole

process manually, hence increasing the possibility of error.

- **No Automated Generation:** Currently, there is no system to automatically create a non-conflict timetable according to the needs of our institution.
- **One-Size-Fits-All Views:** It makes no sense to make a timetable which will not satisfy everyone. Students need to search in this timetable, and teachers cannot quickly find their schedules.
- **Resource Management:** Administrators lack a simple way of knowing which classrooms are vacant at certain times and how effectively our resources are being used.
- **Static Distribution:** After publishing the timetable in pdf form, updating it means distributing it all over again.

Thus, our objective was to develop a system which could help close the chasm between unstructured spreadsheet formats and structured data, schedule creation, editing tools, and provide role-based visualization of the timetable.

1.3 Objectives of the Study

We set ourselves seven concrete objectives:

1. **Heuristic Data Ingestion:** Design an engine that will parse IIITA's complicated Excel timetable, extract metadata (course subjects, instructors, class venues), merge cells, and normalise into a relational schema.
2. **Real-Time Clash Detection:** Design a set of algorithms for detecting conflicts related to time, room availability, instructor workload, and student sections immediately upon importing timetable data.

3. **Role-Based Views:** Design role-based UI interfaces for different actors in the process including:
 - Student Interface with schedule filtering options and electives.
 - Professor Interface with personal schedule only visible to the professor.
 - Admin Interface with master schedule, free room finder, and other administrative tools.
4. **Export and Reports:** Provide users with options to export schedule as PDF, as well as providing Google Sheets support for administrative purposes.
5. **Automated Timetable Generation:** Implement a $(1 + 1)$ Evolution Strategy solver that evaluates candidates against 10 hard constraints and 3 soft constraints, adapts its step size via the $1/5$ th success rule, supports warm-starting from existing schedules, and handling inter-semester constraints through locked sessions.
6. **Interactive Timetable Editor:** Develop a graphical drag-and-drop editor for manual timetable optimisation with instant conflict detection, room and professor modification capabilities, and one-click Large Neighbourhood Search (LNS) auto-resolution for any remaining conflicts.
7. **Intelligent Generator:** Create a guided, multi-step workflow that guides administrators through cluster selection, home room mapping, professor assignment, and solver configuration.

1.4 Scope and Limitations

The scope of the project is to implement all phases of the IIITA Timetabling System, including the React + TypeScript front end, the PostgreSQL back end with Supabase, and the browser-based execution of the (1+1)-ES. Even though the system has been designed specifically for the timetable of IIIT Allahabad, it is still extensible for other colleges.

That said, there are a few important limitations to keep in mind:

- **Parser Sensitivity:** Our Excel parser relies on heuristic rules tuned to IIITA's template. Major deviations from that format may require tweaking the parsing logic.
- **Single-Threaded Solver:** The solver runs on the browser's main thread using batched `setTimeout` calls. This strategy will work well with the present input size, though in case of very large inputs it may become a performance bottleneck.
- **Heuristic Optimality:** Evolution Strategies are meta-heuristics. They guarantee feasibility by providing solutions with zero hard constraint violations, though there is no assurance of optimality for soft constraints based solutions. This depends upon the number of generations.

1.5 Organization of the Thesis

The rest of this thesis is organised as follows:

Chapter 2: Literature Review will provide a review of current researches into university timetabling, including the basic theories of UTP as an NP-hard problem and current algorithmic solutions in literature.

Chapter 3: Technologies and Frameworks will present our choice of technology stack for this project, i.e., React 19, TypeScript, Tailwind CSS, Vite, and Supabase, along with the reasons for their adoption.

Chapter 4: System Architecture and Design will detail the database schema, the data ingest pipeline, the solver architecture, and finally, the client-side routing of components.

Chapter 5: Implementation and Results will form the core of this thesis, where the implementation details of the Excel parser, clash detection algorithms, (1+1)-ES solver and constraint system, drag-and-drop editor and auto-LNS fix, and the intelligent generator will be described.

Chapter 6: Conclusion and Future Work sums up what we achieved, reflects on the challenges we faced, and outlines future directions such as parallelised solvers and hybrid meta-heuristics.

Chapter 2

Literature Review

It is not a new phenomenon that the problem of designing schedules that have no conflicts and that are perfect has received much attention through decades of research. This chapter summarizes the current literature on the UTP, categorizes the different types of algorithms developed by researchers for tackling the problem, and discusses the evolution of software architecture in the context of timetable management systems.

2.1 The University Timetabling Problem (UTP)

Formally speaking, academic scheduling is referred to as the problem of allocating a collection of activities (like lectures, examinations, and meetings) to a collection of resources (time periods, rooms, and teaching staff) based on a variety of constraints [schaerf1999survey]. It should be noted that the difficulty of such a task depends on many factors, such as the scale of the institution, the flexibility of the course structure, and the constraints' strictness.

2.1.1 Constraint Satisfaction and Complexity

In computational complexity theory, the University Timetabling Problem is generally classified as NP-hard [even1975complexity]. This means that when the number of variables

(classes, rooms, teachers) increases, the time needed to solve the problem increases exponentially, making any exhaustive search algorithm totally impractical.

Constraints within the UTP are universally divided into two categories:

- **Hard Constraints:** These are non-negotiable conditions that must be satisfied for a timetable to be considered feasible. Examples are:
 - A room cannot host more than one class simultaneously.
 - A professor cannot teach two different classes at the same time.
 - A student section cannot be scheduled for two different lectures simultaneously.
 - The seating capacity of an assigned room must meet or exceed the number of students enrolled in the class.
- **Soft Constraints:** These represent preferences that enhance the quality of the timetable but are not strictly necessary for its execution. A penalty score is usually assigned when soft constraints are violated, and the goal of optimization algorithms is to minimize this score. Examples include:
 - Minimizing the number of free periods (gaps) in a student's daily schedule.
 - Ensuring faculty members do not have heavily fragmented schedules across the week.
 - Assigning classes to rooms preferred by the instructor.
 - Grouping specific core subjects together sequentially.

2.2 Algorithmic Approaches to Timetabling

Over the years, researchers have proposed a vast array of techniques to solve the UTP, ranging from classical mathematical modeling to modern meta-heuristic algorithms.

2.2.1 Exact Methods

The earliest methods concerned at the mathematical level whether Integer Linear Programming (ILP) and Graph Coloring. In a Graph Coloring model, events are represented as nodes, and edges are drawn between events that conflict (e.g. share the same student group, professor). The objective is to use the fewest possible colours to colour the nodes, where colours indicate distinct time slots, such that no two adjacent nodes are coloured the same colour. Although theoretically valid, exact approaches have significant difficulties due to the high dimension of real-life timetabling problems, which are often unable to find solutions in acceptable computation times when soft constraints are added.

2.2.2 Heuristic and Meta-Heuristic Algorithms

Because exact methods could not deal with large-size problems, heuristics and meta-heuristics became a prominent focus of the scientific community during the late 1990s and 2000s. The methods do not guarantee the absolute best solution, but they are very good at finding “good enough” (feasible and high-quality) solutions in reasonable time.

- **Genetic Algorithms (GA):** Inspired by natural evolution, GAs represent potential timetables as "chromosomes." Through processes of selection, crossover (combining parts of two schedules), and mutation (randomly altering a scheduled event), populations of timetables evolve over generations to minimize constraint violations [burke1996genetic].

- **Simulated Annealing (SA) and Tabu Search:** These local search techniques iteratively make small changes to a complete schedule. Tabu search uses a memory structure (a "tabu list") to prevent the algorithm from revisiting recently evaluated schedules, thereby avoiding getting trapped in local optima [glover1997tabu]. Simulated Annealing mimics the metallurgical process of annealing, occasionally accepting "worse" solutions early in the process to explore a broader search space before gradually "cooling" down to focus on strict optimization [kirkpatrick1983optimization].

2.3 Evolution of Timetable Management Systems

Although a lot of work has been done by researchers to understand schedule generation mathematically, it cannot be ignored that software engineering for timetable management has advanced considerably, in tandem with advances made in software engineering as a whole.

2.3.1 Legacy Systems: Spreadsheets and Desktop Applications

Timetables were traditionally hand-drawn on large whiteboards and grids. The rise of personal computing gave birth to spreadsheet programs such as Microsoft Excel, which became the standard in the academic world. Spreadsheets provide an immensely versatile user interface that is easy to read and understand. As observed by many educational technology experts, however, spreadsheets are inherently static in nature. They are unable to incorporate relational data models that would allow referential integrity enforcement and detection of logical inconsistencies without relying on error-prone, custom-built

macros (e.g., Visual Basic for Applications) [panko1998what]. The problem with sharing an Excel document is the static nature of its data.

Desktop application solutions emerged in the late 1990s and early 2000s as a solution to the drawbacks associated with spreadsheet software. These systems incorporated relational database management systems such as MySQL or Oracle for storing schedule information, providing superior data integrity compared to flat file storage. Nevertheless, due to the desktop-centric nature of such applications, they also had the same distribution problems. The stakeholders involved, namely students and instructors, could not directly communicate with the software but had to resort to printed documents or HTML outputs produced by administrators.

2.3.2 Modern Web-Based Systems

Cloud computing and contemporary web frameworks have greatly influenced the way institutions manage their logistics through software systems. Modern systems employ full-stack web-based architectures, enabling the storage of data centrally while ensuring universal access using web browsers.

With the use of modern front end frameworks such as React, Angular, or Vue.js, highly interactive Single Page Applications (SPAs) have become possible. In contrast to legacy web applications that necessitated full-page refreshes every time an action is performed by a user, SPAs allow for highly fluid user interactions. This is especially important in the case of timetable interfaces since the user needs to be able to change views quickly, for example, switching from "Room view" to "Professor view" and filtering for particular criteria (e.g., only particular students' sections).

In addition, Backend as a Service (BaaS) providers such as Firebase or Supabase have sped up this process considerably. BaaS platforms remove the burden of setting up servers, creating databases, and securing authentications which allows developers to focus on implementing functionality such as conflict detection and heuristic Excel parsers.

2.4 Identification of Gaps in Existing Solutions

While there is an abundance of off-the-shelf schedulers and extensive academic literature on constraints-based scheduling, it seems that there is still a significant gap in many mid-size to large academic institutions, especially those based in developing countries.

1. **The Ingestion Barrier:** Most commercial software applications demand manual data entry via their interface or importing the data from rigidly defined CSV files. Any institution that maintains a more elaborate, unique Excel format (including merged cells, cell coloring, and embedded notes) will face major difficulties converting its current workflow to such a rigid system. There is an urgent need for heuristic parsers that can read "human-written" spreadsheets.

2. **Inadequate Building Occupancy Tools:** Most timetabling applications only produce the desired schedule output (student/faculty). However, no tools are available for campus managers to visualize building occupancy. A "Room Finder" utility or "Master Occupancy Map" is not typically offered as part of the core scheduling engine.

3. **Disconnect between Generation and Management:** The highly scientific solutions provided in academia for scheduling, which are based on Genetic Algorithms, are characterized by terrible interfaces which are not usable by management teams. On the other

hand, well-designed commercial products usually present a fixed schedule that is to be viewed, instead of solved.

The literature therefore suggests the following path: pure algorithmic methods devoid of any consideration of human factors, local spreadsheet-based methods, and cloud-based integrated systems.

TABLE 2.1: Comparison of Timetabling Methodologies

Feature	Spreadsheets (Legacy)	Pure Algorithms	Modern Cloud Sys- tems
Conflict Detection	Manual & Error- Prone	Fully Automated	Real-time Auto- mated
Data Accessibility	Offline / Localized	Varies (often CLI based)	Cloud-synchronized (Web/Mobile)
Flexibility	High (free-form data entry)	Low (strict mathe- matical constraints)	High (heuristics + constraints)
Integration	None	Low	High (APIs, Database links)
User Interface	Basic Grid	Academic / Non- existent	Role-based Dash- boards

The system designed in this thesis is based on this framework, employing the "Modern Cloud System" framework while incorporating a heuristic parser to overcome the limitations of legacy spreadsheet systems, creating an architectural framework that enables seamless automation of constraint solving systems in the future.

Chapter 3

Technologies and Frameworks

The IIITA Timetabling System utilizes a modern, browser-based stack selected to prioritize high-performance data processing and strict referential integrity. This chapter outlines the technical foundation, focusing on tools that facilitate complex client-side computation and the normalization of legacy institutional data.

TABLE 3.1: Summary of the Technology Stack

Layer	Technology	Primary Function in TTGen
Frontend Framework	React 19	Rendering dense, interactive 5-day time grids.
Programming Language	TypeScript	Enforcing strict data contracts for slots and genes.
Database	PostgreSQL	Maintaining integrity across rooms, faculty, and subjects.
Backend / BaaS	Supabase	API gateway and secure database interfacing.
Excel Parsing	SheetJS (xlsx)	Heuristic extraction of semantic meaning from legacy files.

3.1 Frontend Architecture: React 19 and TypeScript

The UI is designed to manage high-density data interactions without performance degradation. To achieve this, the system leverages React’s virtual DOM and TypeScript’s static typing.

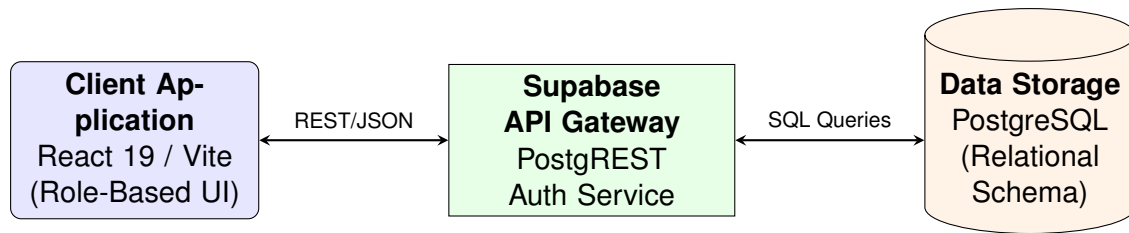


FIGURE 3.1: High-Level System Architecture and Component Interaction

3.1.1 Performance and Data Reliability

The system employs the `useMemo` hook to memoize expensive computations—such as $O(N^2)$ clash detection—ensuring these logic-heavy tasks only execute when the underlying timetable data changes. TypeScript is used to define rigid data interfaces, such as the `ParsedSlot`, ensuring the heuristic Excel parser outputs structured, predictable data.

```
// Example TypeScript Interface for Solver Data Contracts
```

```
interface ParsedSlot {  
  
  id: string;  
  
  day: string;  
  
  startTime: string;  
  
  subjectCode: string;  
  
  type: 'Lecture' | 'Tutorial' | 'Practical';  
  
  roomName: string;  
  
  section: string;  
  
}
```

3.2 Build and Styling: Vite and Tailwind CSS

The development workflow and visual layer are optimized for rapid iteration and dynamic feedback.

- **Vite:** Leverages native ES Modules for near-instantaneous hot-module replacement (HMR), critical during the refinement of grid-generation algorithms.
- **Tailwind CSS:** A utility-first framework that allows the system to dynamically apply visual alerts, such as red ring borders for clashing sessions, directly within the React components.

3.3 Backend and Data Ingestion: Supabase

The system replaces static files with a centralized PostgreSQL database hosted on Supabase, ensuring referential integrity through foreign key constraints.

- **Heuristic Parsing:** The `xlsx` (SheetJS) library is used to read raw institutional spreadsheets. These are processed by a custom engine to extract semantic meaning, such as L-T-P credit structures and complex faculty assignments.
- **Authentication:** Secure access for administrators is managed natively by Supabase Auth, with state handled via a global `AuthContext` provider.

3.4 Solver Engine: Pure TypeScript Architecture

The core contribution of this system is its pure-TypeScript solver engine, which runs directly in the browser. This eliminates the need for server-side optimization and allows for real-time schedule generation. The solver is organized into discrete submodules:

- `dataPrep.ts`: Expands high-level credits into individual sessions and applies the 2+1 lecture format.
- `constraints.ts`: Evaluates solutions against ten hard and three soft constraints.

- `mutations.ts`: Implements five mutation operators, including day reassignment and Gaussian time shifts.
- `solver.ts`: Manages the (1+1)-ES loop, elitist selection, and stagnation restarts.
- `localSearch.ts`: Provides the LNS auto-fix routine for the interactive editor.

3.5 External Integrations: Reporting and Export

To support administrative workflows, the system integrates with external reporting tools:

- **Google Sheets API**: Exports formatted schedules with frozen headers and color-coded session types directly to Google Drive.
- **PDF Generation**: Captures high-fidelity snapshots of the timetable grid using `html-to-image` and `jsPDF` for immediate offline use.

Chapter 4

System Architecture and Design

This chapter details the structural design of the IIITA Timetabling System. It defines the application topology, the relational database schema, the solver's internal data model, the asynchronous processing design for non-blocking execution, the Excel data ingestion pipeline, and the state-based routing logic used for view management.

4.1 Application Topology

The system architecture consists of three primary components:

- **Frontend (React SPA):** A Single Page Application that handles user interaction, clash detection, and PDF generation. It interacts with the backend through REST API calls.
- **Backend (Supabase):** Provides the API gateway, manages user authentication, and handles database interactions.
- **Database (PostgreSQL):** A relational database that stores institutional data, including users, rooms, subjects, faculty, and the generated timetable slots.

4.2 Database Schema Design

The database schema is normalized to ensure data integrity and minimize redundancy. It consists of eight tables: six core entity tables and two mapping tables for the solver engine. Figure 4.1 illustrates the Entity-Relationship (ER) diagram.

4.2.1 Core Entities

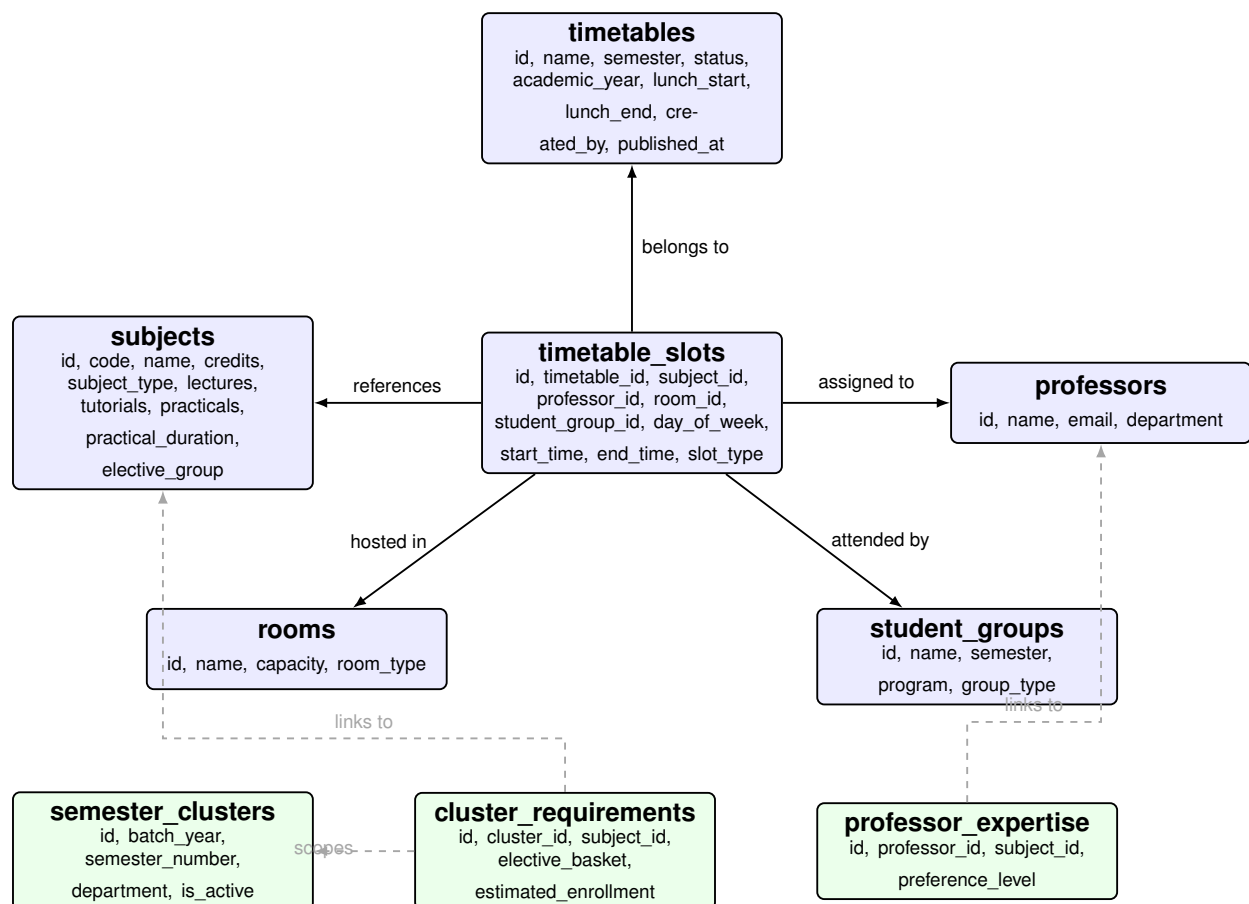


FIGURE 4.1: Entity-Relationship Diagram of the Complete Database Schema

- **timetables:** The container for a specific schedule. It stores the iteration name, semester, academic year, and lunch break constraints.
- **subjects:** Contains course metadata, including the course code, credit structure (L-T-P), and elective grouping.
- **professors:** Stores faculty information, uniquely identified by email.

- **rooms:** Physical room data including capacity and room type (Lecture Hall or Laboratory).
- **student_groups:** Defines student cohorts by program, semester, and section.

4.2.2 The Central Hub: `timetable_slots`

The `timetable_slots` table is the primary junction table connecting timetables to subjects, professors, rooms, and student groups. Table 4.1 details its structure.

TABLE 4.1: Schema Specification for `timetable_slots`

Column Name	Data Type	Description & Constraints
<code>id</code>	UUID	Primary Key, auto-generated.
<code>timetable_id</code>	UUID	Foreign Key referencing <code>timetables</code> .
<code>subject_id</code>	UUID	Foreign Key referencing <code>subjects</code> .
<code>professor_id</code>	UUID	Foreign Key referencing <code>professors</code> .
<code>room_id</code>	UUID	Foreign Key referencing <code>rooms</code> .
<code>student_group_id</code>	UUID	Foreign Key referencing <code>student_groups</code> .
<code>day_of_week</code>	Integer	1 = Monday, ..., 5 = Friday.
<code>start_time</code>	VarChar	24-hour format (e.g., "09:00").
<code>end_time</code>	VarChar	24-hour format (e.g., "11:00").
<code>slot_type</code>	VarChar	Enum: 'Lecture', 'Tutorial', 'Practical'.

To support automated generation, the schema includes mapping tables used by the solver engine:

- **semester_clusters:** Groups subjects by batch, semester, and department to define the scope of a scheduling task.
- **cluster_requirements:** Maps subjects to clusters, specifying elective groups and estimated enrollment for room allocation constraints.

- **professor_expertise:** Defines which subjects a professor can teach and their associated preference levels.

4.2.3 Solver Data Structures

The solver engine operates on a specific set of TypeScript interfaces that represent the scheduling problem as a Genetic Algorithm chromosome. Figure 4.2 shows the relationships between these core interfaces.

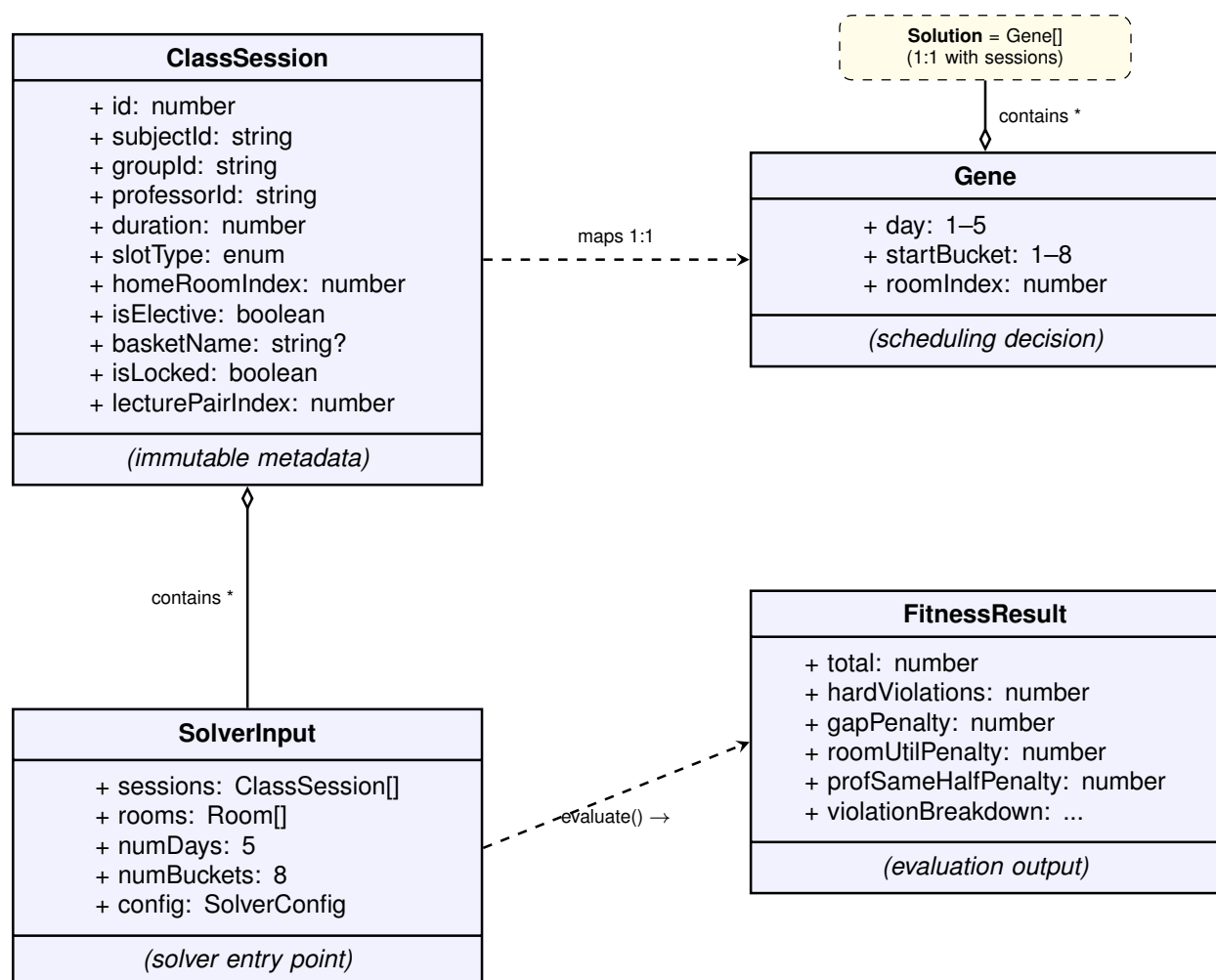


FIGURE 4.2: UML Class Diagram of the Solver Data Model

4.3 Asynchronous Processing Design

To execute the (1+1)-ES solver without blocking the browser's single-threaded UI, the following strategy is used:

- **Single-Threaded Execution:** Since JavaScript runs on a single main thread, a continuous (1+1)-ES loop would block UI updates and result in "unresponsive script" warnings.
- **Asynchronous Batched Execution:** The solver partitions the optimization workload into discrete batches of generations, preventing the CPU from becoming an uninterruptible bottleneck.
- **Event Loop Yielding:** After each batch, the system utilizes `setTimeout(..., 0)` to yield control back to the browser's event loop, allowing for essential tasks like input handling.
- **UI Responsiveness:** This non-blocking design enables the application to render real-time fitness charts and maintain a fluid, interactive user experience throughout the optimization process.

The system includes a data ingestion module (`TimetableImporter`) to parse existing Excel-based timetables into the normalized database schema. The process follows a five-step pipeline:

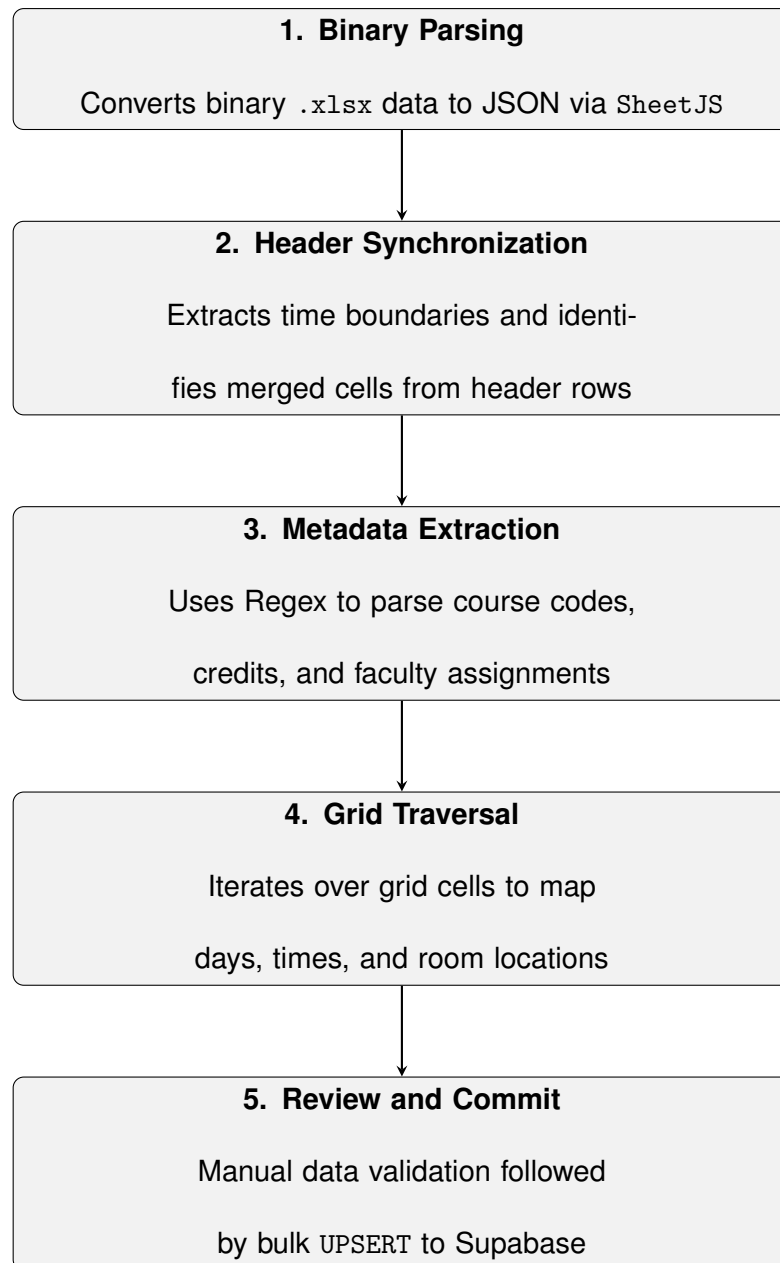


FIGURE 4.3: Data Ingestion Pipeline for Excel Timetables

1. **Binary Parsing:** Uses the SheetJS library to read '.xlsx' files and convert them into JSON arrays.
2. **Header Synchronization:** Identifies time-slot boundaries from the header row, supporting various time formats and identifying merged cells for multi-hour sessions.
3. **Metadata Extraction:** Uses Regular Expressions to identify subject codes, faculty

names, and credit structures. It handles complex strings such as split-section assignments.

4. **Grid Traversal:** Maps grid cells to specific days and times, extracting room and section information.
5. **Review and Commit:** Provides an interface for manual correction before performing bulk UPSERT operations to Supabase.

The application uses state-based view management rather than traditional URL routing.

A central 'currentView' state in the root component determines the active module.

- `import`: Mounts the `TimetableImporter`.
- `generator`: Mounts the multi-step `GeneratorView` wizard for automated timetable creation.
- `editor`: Mounts the interactive `EditorView` for drag-and-drop timetable refinement with LNS auto-fix.
- `student`: Mounts the `TimetableViewer` (All Students administrative view).
- `prof`: Mounts the `ProfessorTimetableViewer`.
- `room`: Mounts the `RoomTimetableViewer`.
- `master-room`: Mounts the campus-wide `MasterRoomViewer`.
- `free-room`: Mounts the `FreeRoomViewer`.
- `report-card`: Mounts the `ReportCardView`.

This approach ensures that only the logic and data required for the active module are loaded. Custom hooks (`useGeneratorData`, `useEditorData`) manage data fetching and caching, separating the data layer from the UI components.

Chapter 5

Implementation and Results

5.1 Introduction to the Optimization Engine

The transition from data ingestion to the core solver introduces a highly constrained NP-hard optimization problem. The IIITA Timetabling System models this challenge mathematically, solving it via client-side heuristics. Rather than relying on traditional deterministic approaches which fail to scale, the system employs an evolutionary search mechanism. The engine processes high-dimensional inputs—comprising student cohorts, faculty assignments, and spatial capacities—and systematically navigates a vast, discontinuous search space to identify a globally optimal, conflict-free schedule.

5.2 Mathematical Representation of the Schedule

To facilitate computational optimization, the schedule is encoded into a specific data structure known as a *Chromosome*. The solver models a complete timetable S as an ordered array of *genes*. Each gene $g_i \in S$ encodes the precise scheduling decision for the i -th class session, mapping it to a spatial and temporal coordinate.

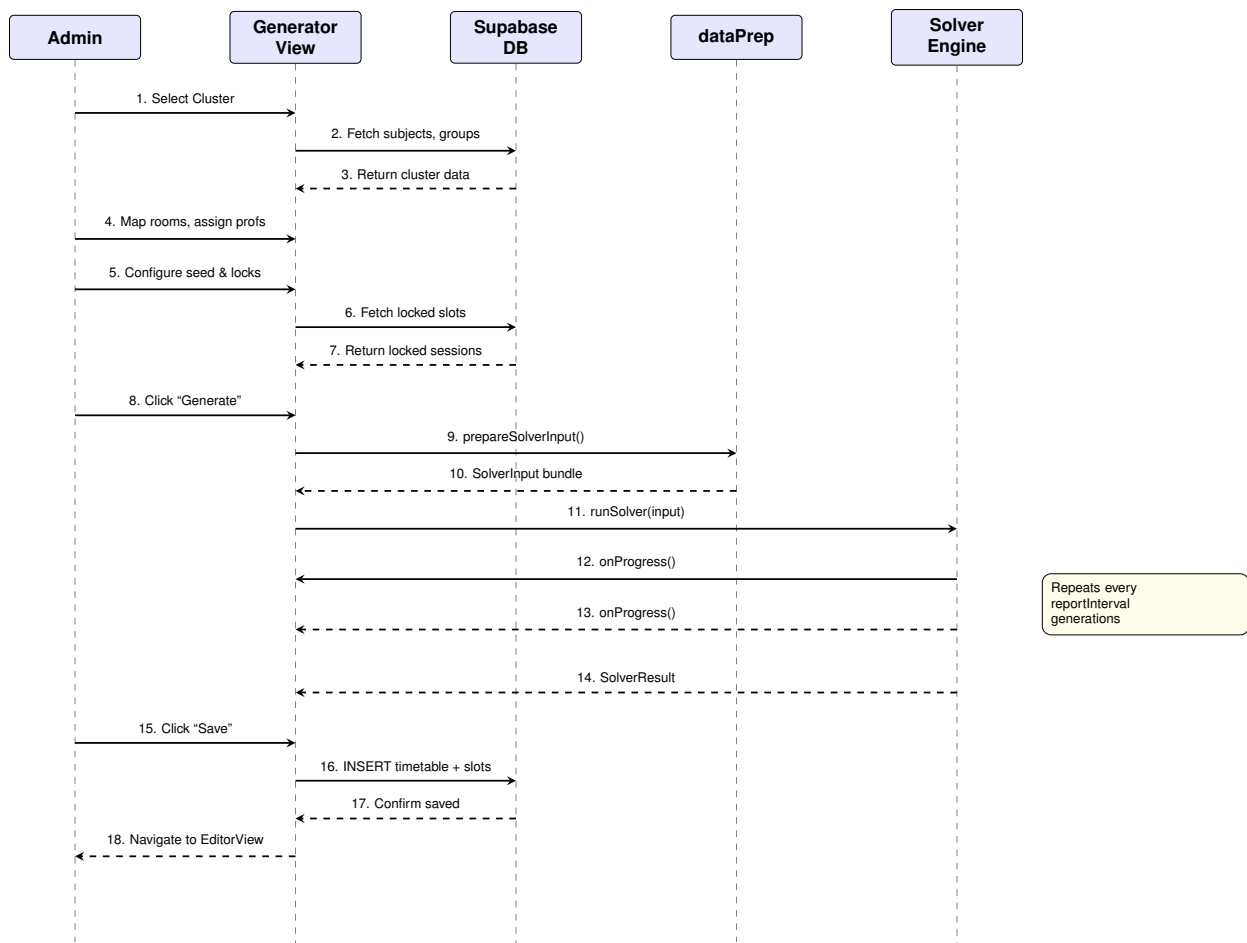


FIGURE 5.1: Sequence Diagram: End-to-End Intelligent Generator Workflow

Formally, a gene is defined as a tuple:

$$g_i = (d_i, t_i, r_i) \quad (5.1)$$

where:

- $d_i \in \{1, 2, 3, 4, 5\}$ represents the day of the week (Monday through Friday).
- $t_i \in \{1, 2, \dots, 8\}$ represents the discrete start time slot (each slot corresponding to one hour, from 08:50 to 18:30).
- $r_i \in \mathcal{R}$ is the index into the array of available physical rooms.

Each gene is invariably linked to an immutable `ClassSession` object that contains the session's metadata: the specific subject identifier, the student group G_i , the assigned professor p_i , the duration D_i , and the specific slot type (Lecture, Tutorial, or Practical).

5.3 Mathematical Formulation of Constraints

The optimization objective is governed by a composite fitness function that evaluates both the feasibility and the quality of the candidate solution S .

5.3.1 Hard Constraints (The Feasibility Space)

Hard constraints define the boundary of feasible solutions. A schedule is strictly invalid if any hard constraint is violated. For two distinct sessions i and j , we define the temporal

overlap condition as:

$$\text{overlap}(t_i, t_j) = \begin{cases} 1 & \text{if } \max(t_i, t_j) \leq \min(t_i + D_i - 1, t_j + D_j - 1) \\ 0 & \text{otherwise} \end{cases} \quad (5.2)$$

The engine enforces the following critical hard penalties:

- **Professor Overlap Penalty:** A professor p cannot be physically present in two different locations at time t .

$$\sum_{i \neq j} I(p_i = p_j \wedge d_i = d_j \wedge \text{overlap}(t_i, t_j)) = 0 \quad (5.3)$$

- **Student Group Overlap Penalty:** A specific student cohort G cannot attend two distinct subjects simultaneously. (Concurrent sessions from the same synchronized elective basket are mathematically exempted).

$$\sum_{i \neq j} I(G_i = G_j \wedge d_i = d_j \wedge \text{overlap}(t_i, t_j) \wedge \neg \text{SameBasket}(i, j)) = 0 \quad (5.4)$$

- **Room Clash Penalty:** A physical room r can host at most one session at any given time.

$$\sum_{i \neq j} I(r_i = r_j \wedge d_i = d_j \wedge \text{overlap}(t_i, t_j)) = 0 \quad (5.5)$$

- **Lunch Boundary Enforcement:** Sessions cannot cross the designated lunch break (B_{lunch} from 13:00 to 14:30, bounded between slot 4 and 5).

$$\forall i \in S, \quad [t_i, t_i + D_i - 1] \cap B_{lunch} = \emptyset \quad (5.6)$$

- **L-T-P “Chunking” (The 2+1 Rule):** For core courses with $L \geq 2$ credits, the algorithm mathematically forces the contact hours into a contiguous 2-hour block and subsequent 1-hour blocks, distributed across distinct days. Let $\mathcal{L}_c$ be the set of lecture sessions for subject c :

$$\exists! k \in \mathcal{L}_c : D_k = 2 \quad \text{and} \quad \forall j \in \mathcal{L}_c \setminus \{k\}, D_j = 1 \quad (5.7)$$

$$\forall i, j \in \mathcal{L}_c, i \neq j \implies d_i \neq d_j \quad (5.8)$$

TABLE 5.1: Hard Constraints Enforced by the Solver

#	Constraint Name	Description
1	Time Boundary	A session must not extend beyond slot 8 (18:30).
2	Break/Lunch Crossing	A multi-slot session must not span across a break boundary (slots 2→3 or 4→5).
3	Room Non-Overlap	At most one session may occupy a given room at any (day, slot).
4	Professor Non-Overlap	A professor can teach at most one session at any given slot. No elective exemption.
5	Group Non-Overlap	A student group can attend at most one session per slot. Elective–elective pairs and same-basket pairs are exempt.
6	Elective Basket Sync	Synced baskets (HSMC, MDM) must share the same (day, slot). No two different baskets may share a slot (global anti-clash).
7	Lab Room	Practical sessions must be assigned to rooms with type “Lab”.
8	WMC-Section Overlap	A whole-batch (WMC) session cannot occupy the same slot as any section-level session.
9	Home Room	Core lectures and tutorials must be placed in their assigned home room.
10	2+1 Lecture Format	The double-lecture block and single-lectures for the same subject+group must be on different days.

5.3.2 Soft Constraints (The Quality Space)

Once the solution space is feasible, the engine optimizes for schedule quality by minimizing soft penalties.

- **Minimizing Student Gaps:** The engine minimizes idle slots between a cohort's first and last class on any given day. Let $\mathcal{T}_{g,d}$ be the set of all active time slots for student group g on day d . The gap penalty $G(S)$ is:

$$G(S) = \sum_g \sum_d ((\max(\mathcal{T}_{g,d}) - \min(\mathcal{T}_{g,d}) + 1) - |\mathcal{T}_{g,d}|) \quad (5.9)$$

- **Room Capacity Optimization:** The system matches class enrollment c_i to room capacity K_{r_i} . Let the utilization ratio be $u_i = c_i / K_{r_i}$. Given optimal thresholds $\alpha = 0.6$ and $\beta = 1.2$, the room utilization penalty $R(S)$ is formulated as:

$$R(S) = \sum_i (\max(0, \alpha - u_i) + \max(0, u_i - \beta)) \quad (5.10)$$

5.4 The (1+1) Evolution Strategy Solver

The optimization engine is powered by a (1+1) Evolution Strategy (ES) algorithm. It relies on iterative stochastic perturbations and adaptive mutation rates to navigate the constrained schedule space.

5.4.1 Strict Elitism

The algorithm employs a strictly elitist selection mechanism. In each generation, a parent solution S_{parent} undergoes mutation to produce a single offspring S_{child} . The offspring

replaces the parent if and only if its total fitness score is less than or equal to the parent's score:

$$S_{parent}^{(t+1)} = \begin{cases} S_{child} & \text{if } f(S_{child}) \leq f(S_{parent}^{(t)}) \\ S_{parent}^{(t)} & \text{otherwise} \end{cases} \quad (5.11)$$

This greediness guarantees that the solver never loses ground on hard constraint feasibility.

5.4.2 Mutation Operators

To generate S_{child} , the engine selects a subset of mutable sessions and applies highly stochastic operators:

- **Gaussian Time Shifts:** The start time t_i is perturbed using Gaussian noise scaled by variance σ :

$$t_i^{(new)} = t_i + [\mathcal{N}(0, \sigma^2)] \quad (5.12)$$

- **Type-Aware Room Reassignments:** A session is moved to a randomly selected room $r_k \in \mathcal{R}$ that strictly matches its required architectural type (e.g., Lab vs. Lecture Hall).
- **Targeted Swaps:** Two sessions i and j of identical duration exchange their spatial-temporal coordinates, effectively jumping out of local minima without increasing the overall density.

5.4.3 Adaptive Variance (The Brain)

To dynamically balance global exploration and local exploitation, the solver implements Rechenberg's 1/5th Success Rule. The mutation strength σ is adapted every $W = 50$

generations based on the success rate p_s (the ratio of offspring that successfully improved or matched fitness).

$$\sigma \leftarrow \begin{cases} \sigma \times 1.22 & \text{if } p_s > 0.20 \quad (\text{Search wider}) \\ \sigma \times 0.82 & \text{if } p_s < 0.20 \quad (\text{Search finer}) \\ \sigma & \text{otherwise} \end{cases} \quad (5.13)$$

5.4.4 Stagnation Recovery

In highly dense optimization landscapes, the algorithm may converge to a deep local optimum (a plateau). The engine mathematically tracks this stagnation. If S_{parent} fails to improve for 15,000 consecutive generations, a stagnation recovery protocol is triggered. The current candidate is discarded, and maximum entropy is injected by generating a completely new randomized initial timetable.

5.5 Interactive Editor & Large Neighbourhood Search (LNS)

The user interface functions primarily as an engineering tool designed for granular control over the algorithmic output. When manual drag-and-drop actions introduce systemic conflicts, an integrated Large Neighbourhood Search (LNS) is deployed to repair the grid.

The “Auto-Fix” feature applies a destroy-and-repair heuristic. Instead of recalculating the entire grid, it isolates the localized clashing subset $C \subseteq S$. The LNS algorithm iterates up to 1,500 times, randomly selecting a session $i \in C$, applying stochastic mutations

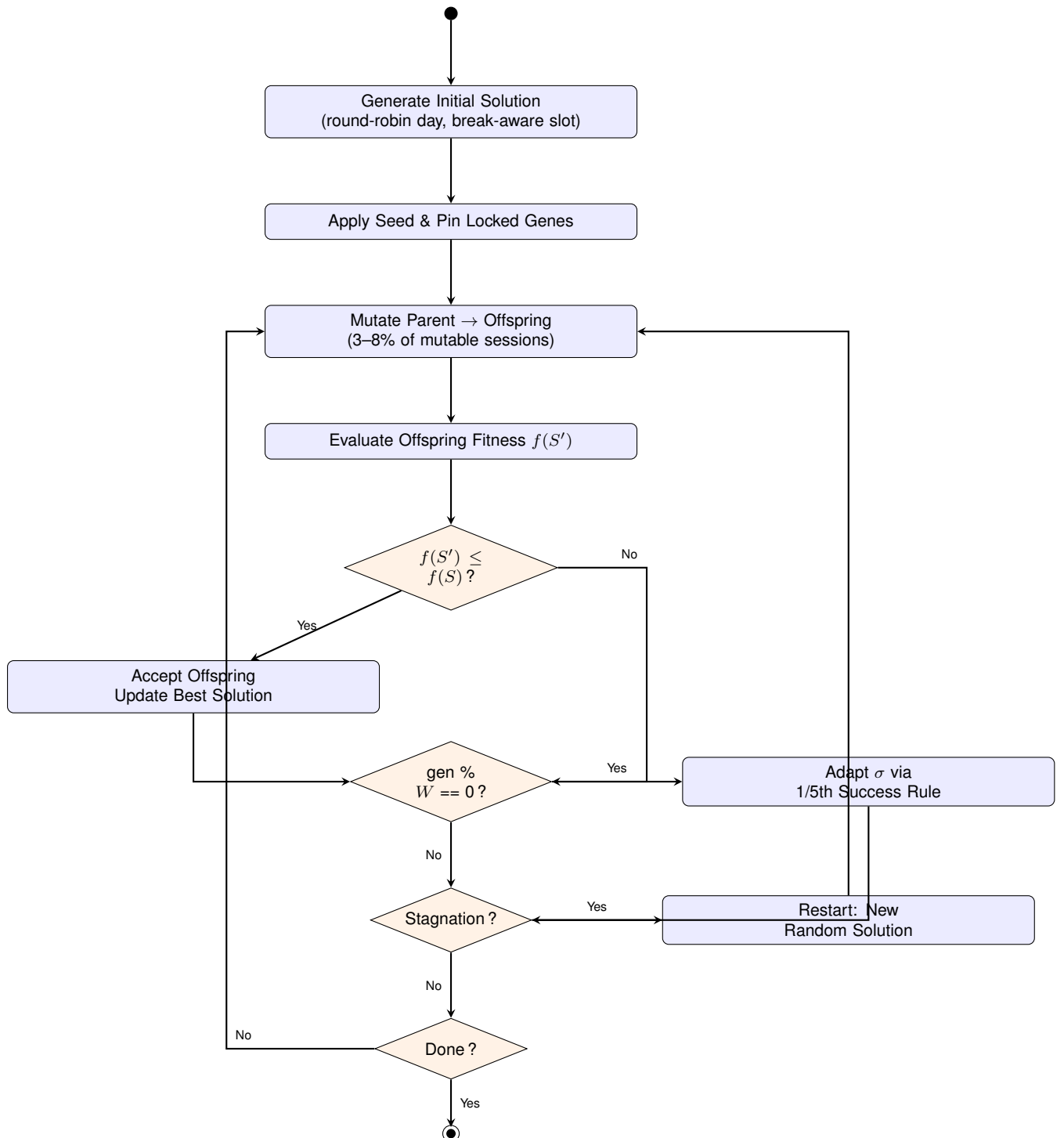


FIGURE 5.2: Activity Diagram: (1+1)-ES Solver Execution Flow

(relocation, time shifts, or swaps), and evaluating the local differential fitness. If a localized mutation resolves the clash without violating surrounding sessions, the repair is committed.

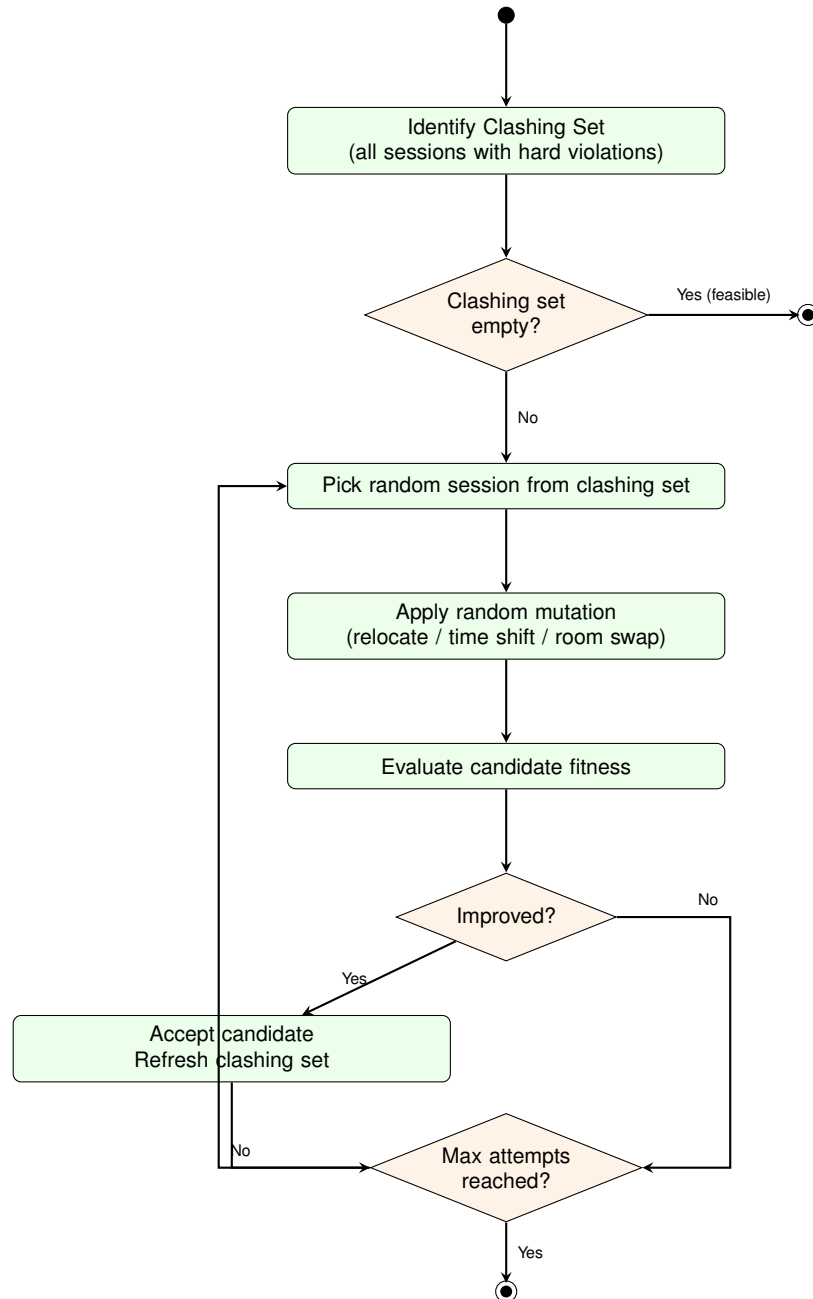


FIGURE 5.3: Activity Diagram: LNS Destroy-and-Repair Auto-Fix

5.6 Results and Application Features

The implementation of the optimization engine drastically reduces the administrative overhead. By executing entirely client-side, the system achieves feasibility rapidly, circumventing backend server latencies.

5.6.1 Algorithmic Convergence

The solver demonstrates rapid convergence. The heavily weighted hard penalties force the evolution strategy to prioritize feasibility before aggressively optimizing the soft quality metrics.

‘[Insert Image: Fitness Score vs. Generations Graph]’

5.6.2 The Interactive Drag-and-Drop Grid

The primary workspace visualizes the temporal matrix. It calculates multi-dimensional clashes in real-time, surfacing spatial and temporal overlaps through distinct programmatic warnings.

‘[Insert Image: Main UI Grid showing Clashes]’

5.6.3 LNS Auto-Fix Execution

When administrators execute the LNS repair routine, the interface explicitly highlights the target neighbourhood and dynamically resolves overlapping nodes without destabilizing the optimized framework.

‘[Insert Image: UI highlighting Auto-Fix suggestions]’

5.6.4 Export and PDF Generation

The final scheduling matrix is programmatically serialized into a high-fidelity PDF, leveraging DOM capture logic to bypass CSS layout constraints, ensuring the complete multi-dimensional array is rendered cleanly.

‘[Insert Image: Final PDF Timetable Output]’

5.7 Dynamic Time Grid Generation

A significant challenge in displaying timetables is the variability of time slots. Hardcoding a static grid (e.g., exactly ten 1-hour slots from 9:00 AM to 7:00 PM) results in wasted visual space when classes end early, and fails to accommodate non-standard time ranges (such as a 1.5-hour class).

To solve this, all viewer components in the system implement a **Dynamic Time Column Generator algorithm**. When a timetable is selected, the system fetches all associated `timetable_slots`. The algorithm then executes the following steps:

1. Extracts all unique `start_time` and `end_time` strings from the raw slot data.
2. Injects hardcoded institutional break times: 10:50 to 11:00 (Morning Break) and 13:00 to 14:30 (Lunch Break).
3. Sorts all extracted timestamps chronologically to create an array of discrete time boundaries.
4. Iterates through adjacent pairs of boundaries to generate standard columns.
5. Filters out any generated columns that fall entirely within the lunch or break periods, or that contain absolutely zero scheduled classes.

The result is a highly responsive grid that only displays columns for active periods, maximizing screen real estate and improving readability.

5.8 Multi-Dimensional Clash Detection Algorithm

One of the most critical administrative features is the automated detection of scheduling conflicts. The ‘TimetableViewer’ component implements a robust $O(N^2)$ algorithm that compares every slot against every other slot within the selected timetable to detect overlaps.

An overlap is mathematically defined as occurring when:

$$(SlotA.start < SlotB.end) \wedge (SlotA.end > SlotB.start) \quad (5.14)$$

When an overlap is detected, the algorithm categorizes the clash into one of four distinct types as summarized in Table 5.2.

TABLE 5.2: Types of Scheduling Clashes Detected

Clash Type	Logical Condition and Description
Room Clash	$SlotA.room == SlotB.room$ Two different classes are assigned to the exact same physical space simultaneously.
Professor Clash	$SlotA.professor == SlotB.professor$ A single faculty member is booked to teach two different classes at the same time.
Section Clash	$SlotA.section == SlotB.section \wedge SlotA.subject \neq SlotB.subject$ A specific student cohort is expected to attend two distinct subjects simultaneously.
WMC-Section	Specialized institutional rule where generic weekend/WMC sessions overlap with standard section sessions.

These clashes are immediately surfaced to the administrator in a dedicated visual panel, complete with distinct iconography (e.g., a Map Pin for Room clashes, a User icon for Professor clashes), allowing for rapid identification and remediation.

5.9 Inverted Logic: The Free Room Finder

Traditional scheduling systems focus on when rooms are occupied. However, students and staff frequently need to know when rooms are *available*. The 'FreeRoomViewer' component achieves this through an inverted logic algorithm.

Unlike the dynamic grid, the Free Room Finder establishes a hardcoded temporal matrix (e.g., 8:50 AM to 6:30 PM in discrete blocks).

The algorithm executes as follows:

1. The system first queries the database for **all** existing rooms.
2. It then queries the `timetable_slots` to retrieve all currently active sessions.
3. For every cell in the temporal matrix (Day $\times$ Time), it instantiates the complete set of all campus rooms.
4. It iterates through the active sessions. If a session falls within the current cell's Day and Time, that session's `room_id` is removed from the set of available rooms.

The remaining rooms in the set are then rendered in the UI as green badges, instantly showing users where they can find empty spaces across the campus.

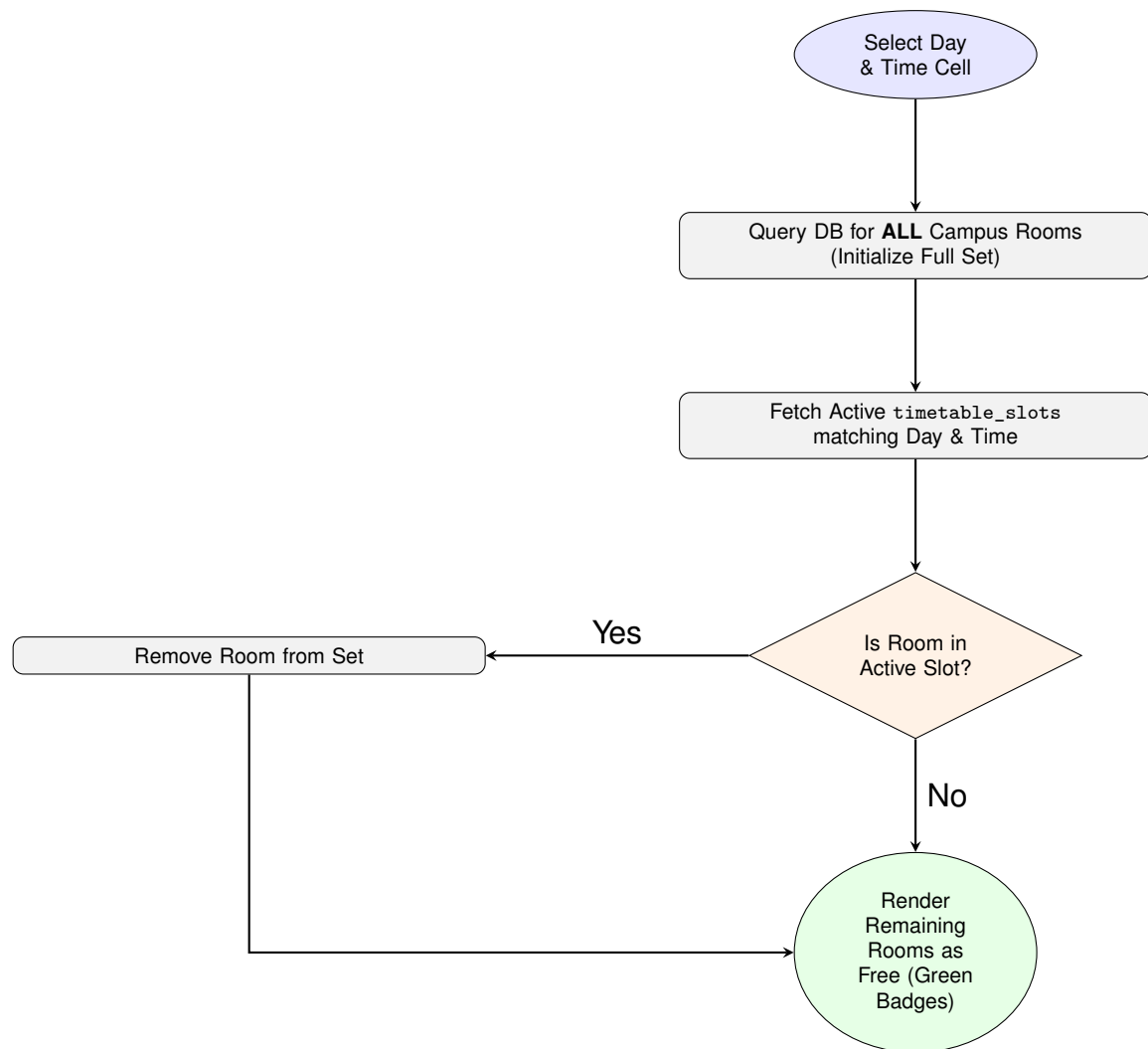


FIGURE 5.4: Inverted Logic Flowchart for the Free Room Finder Algorithm

5.10 React Component Hierarchy

To maintain a scalable and manageable codebase, the frontend relies on a strict component tree hierarchy. Figure 5.5 illustrates the relationship between the global layout and the localized, role-based timetable viewers.

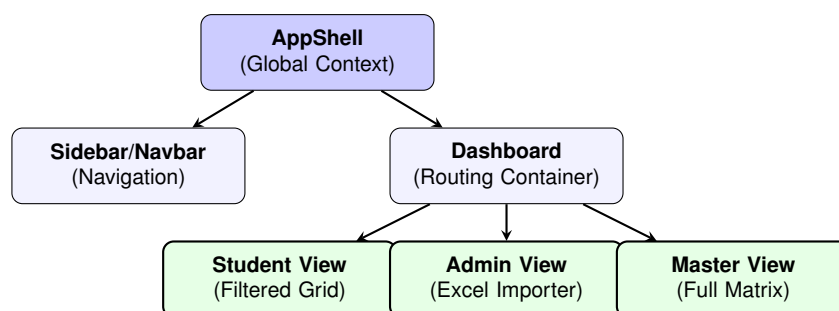


FIGURE 5.5: Hierarchical Structure of the Core React Components

5.11 Role-Based Viewer Implementations

The system provides several specialized views tailored to different stakeholders, summarized in Table 5.3.

TABLE 5.3: Role-Based Component Mapping and Data Scope

Stakeholder Role	Component View	Primary Filter Parameter
Student	<code>TimetableViewer</code>	Semester & Specific Section
Professor	<code>ProfessorTimetableViewer</code>	Faculty Email/ID
Department Admin	<code>RoomTimetableViewer</code>	Specific Room Name
Campus Admin	<code>MasterRoomViewer</code>	Building prefix (e.g., “CC-3”, “LT”)
Facility Staff	<code>FreeRoomViewer</code>	Temporal Matrix (Empty Rooms)

5.11.1 Student Cohort View

Given the complexity of modern curricula featuring numerous electives and minor tracks, a master timetable can be overwhelming. The `TimetableViewer` allows users to filter the

master schedule by selecting a specific semester and section.

Crucially, it implements a **Smart Merging algorithm**. Often, core lectures are held for "All Sections" simultaneously, while practical laboratory sessions are split by specific section (e.g., "Sec A"). The merging algorithm intelligently filters the master data, pulling in slots explicitly assigned to the selected section *and* slots assigned to the generic "All" section (WMC), combining them into a single, cohesive cohort schedule.

5.11.2 Master Occupancy Viewer

For campus administrators, the 'MasterRoomViewer' provides a macro-level perspective. It displays a dense grid where rows represent physical rooms rather than student groups. Because an institution may have hundreds of rooms, this view implements a building-level filter. The system extracts the building prefix from the room name (e.g., identifying "CC-3" from "CC-3-5106") and allows the administrator to filter the grid, rendering a manageable overview of specific campus zones.

5.12 Export Mechanisms and Results

To bridge the gap between the digital system and offline requirements, the platform implements robust export mechanisms.

5.12.1 Client-Side PDF Generation

Users can generate PDFs of their current view. This relies on the 'html-to-image' library to capture the DOM state as a PNG, which is then embedded into a 'jsPDF' document. A significant technical hurdle overcome during implementation was the handling of sticky

CSS headers; the capture logic temporarily disables ‘position: sticky’ properties on the DOM elements right before capture to ensure the entire scrollable area is rendered to the image buffer accurately.

5.12.2 Google Sheets API Export

The ‘RoomTimetableViewer’ and ‘FreeRoomViewer’ support bulk export directly to Google Sheets. Using the Google Identity Services client, the React application constructs a multi-layered batch update request. This request not only writes the raw string data into the cells but explicitly formats the workbook—creating separate worksheets for different rooms, freezing header rows, applying pink and gray background colors to headers, and drawing solid borders—resulting in a highly professional, ready-to-share document directly in the administrator’s Google Drive.

5.12.3 User Interface Evaluation

The resulting application is a highly responsive, modern interface. By utilizing Tailwind CSS, the application maintains a clean, minimalist aesthetic with clear color coding (e.g., orange for practicals, green for tutorials). The elimination of page reloads between views, combined with the instantaneous clash detection, represents a monumental upgrade in usability and efficiency compared to traditional spreadsheet workflows.

5.13 The Interactive Timetable Editor

The `EditorView` component is where human judgment meets algorithmic output. It gives administrators an interactive, drag-and-drop interface for refining timetables—whether those timetables were generated by the solver or imported from Excel.

5.13.1 Drag-and-Drop Slot Rearrangement

The editor renders a dense 5-day $\times$ N -slot grid, where each scheduled session appears as a draggable card. Administrators can drag any session card from one cell to another to reassign its day and start time. The system automatically recalculates the end time based on the session's duration and validates that the new position does not fall in a lunch or break slot. All drag operations push the previous state onto a 30-level undo stack.

5.13.2 Inline Editing

Each session card exposes a hover-activated edit panel that allows inline modification of:

- **Room Assignment:** A cascading two-step selector where the administrator first selects the room type (Lecture or Lab), then picks from filtered rooms of that type.
- **Professor Assignment:** A dropdown populated from the full professor list.

5.13.3 Real-Time Conflict Visualization

The editor implements a client-side clash detection algorithm that runs on every state change. Sessions involved in any conflict are highlighted with a red background and ring border. The algorithm checks the same constraints as the solver: room overlaps, professor overlaps, section overlaps (with elective exemptions), WMC-section conflicts, cross-basket elective clashes, and the 2+1 same-day lecture rule.

5.13.4 Feasibility Check

A dedicated “Check Feasibility” button constructs a full `SolverInput` from the current editor state, converts the slot positions into a `Solution` (Gene array), appends locked sessions from all other published timetables, and invokes the solver’s `evaluate()` function. The result displays the total fitness score and the per-constraint violation breakdown, giving the administrator a precise understanding of remaining issues.

5.13.5 LNS Auto-Fix

When conflicts exist, the administrator can invoke the **Large Neighbourhood Search (LNS)** auto-fix [[shaw1998using](#)]. This routine:

1. Identifies all session indices involved in any hard constraint violation (the “clashing set”).
2. Iterates for up to 1500 attempts. In each attempt, a random session from the clashing set is selected and a random mutation (relocate, time shift, or room change) is applied.
3. If the mutation reduces the total fitness, the improved solution is accepted and the clashing set is refreshed.
4. The process terminates early if all hard violations reach zero.

The LNS targets only the problematic sessions rather than reoptimizing the entire schedule, making it significantly faster than a full solver run while preserving the administrator’s manual adjustments to non-conflicting sessions. Figure 5.6 illustrates this destroy-and-repair process.

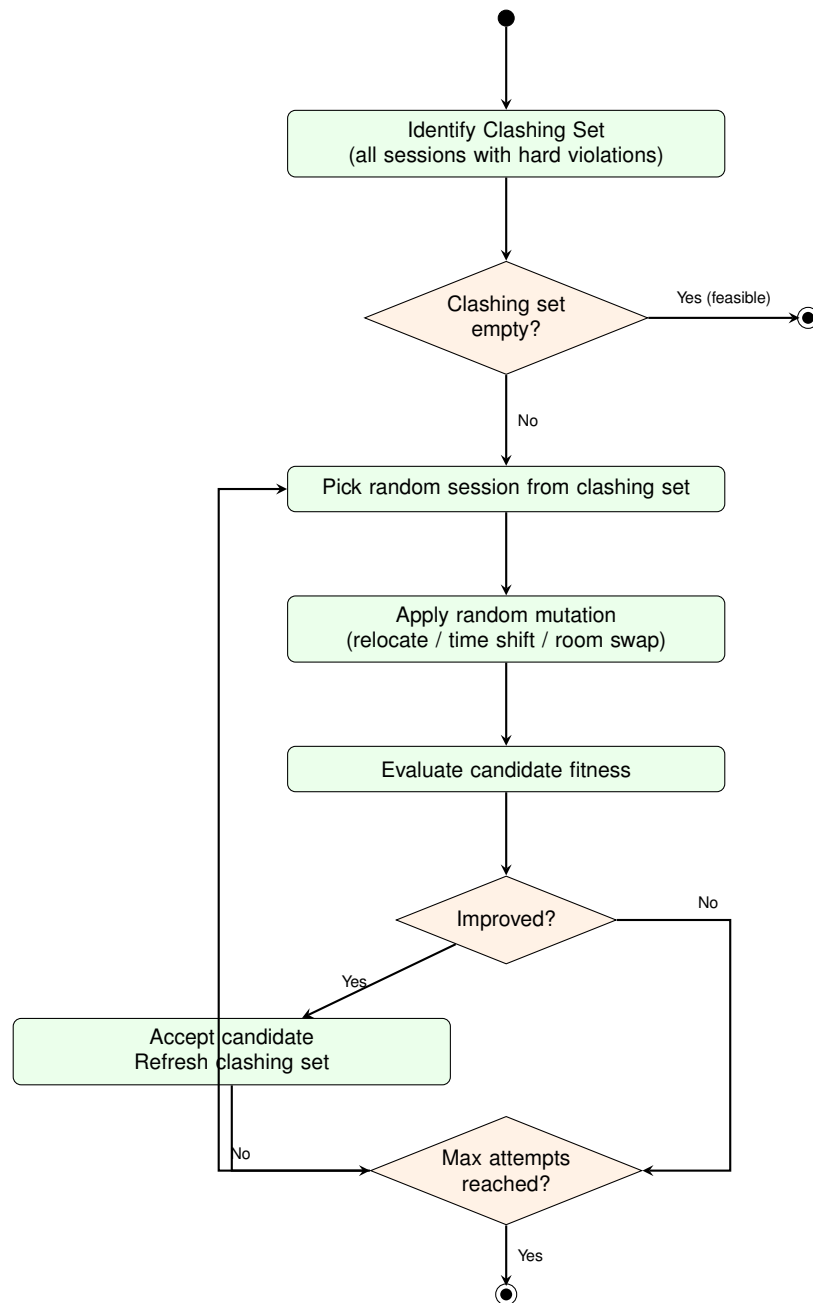


FIGURE 5.6: Activity Diagram: LNS Destroy-and-Repair Auto-Fix

Chapter 6

Conclusion and Future Work

Timetabling touches every person on a university campus, every single day. Getting it wrong means wasted hours, frustrated students, and overworked coordinators. In this thesis, we set out to replace the manual, spreadsheet-driven scheduling process at IIIT Allahabad with something better: a modern, web-based platform that can generate, edit, and distribute timetables intelligently.

6.1 Summary of Contributions

The biggest achievement of this project is taking what used to be a weeks-long manual process and turning it into something that runs in seconds. Here is what we built:

- **Heuristic Ingestion Engine:** We wrote an Excel parser that can read IIITA's messy, human-formatted spreadsheets, pull out the important bits (subjects, faculty, rooms), and load everything into a clean PostgreSQL database. This alone saves hours of manual data entry.
- **Real-Time Conflict Resolution:** The implementation of a multi-dimensional $O(N^2)$ clash detection algorithm that automatically surfaces Room, Professor, Section, and

WMC-Section overlaps. This feature alone significantly reduces the administrative burden of manually verifying schedule integrity.

- **(1+1)-ES Automated Solver:** The implementation of a fully functional timetable generator using a (1+1) Evolution Strategy algorithm. The solver evaluates candidate schedules against a comprehensive fitness function comprising ten hard constraints (room, professor, and group non-overlap; break/lunch boundary enforcement; lab-room assignment; elective basket synchronization; WMC-section isolation; home-room binding; the 2+1 lecture format; and time boundary compliance) and three soft constraints (student schedule gaps, room utilization efficiency, and professor same-half-day avoidance). The algorithm employs Rechenberg's 1/5th success rule for adaptive step-size control and a stagnation restart mechanism.
- **Interactive Timetable Editor:** The development of a drag-and-drop editing interface with real-time conflict visualization, inline room and professor editing, a 30-level undo stack, and a Large Neighbourhood Search (LNS) auto-fix routine that targets clashing sessions. The editor's Master View provides cross-semester constraint awareness by overlaying slots from all published timetables.
- **Multi-Step Intelligent Generator:** The creation of a guided configuration workflow enabling administrators to select semester clusters, auto-map home rooms using a floor-based naming policy, assign professors via an interactive matrix with expertise-based filtering, and configure seed and locked timetables before launching the solver.
- **Cross-Semester Constraint Awareness:** The implementation of locked sessions from published timetables, enabling the solver and editor to detect and avoid room and professor conflicts across independently scheduled semesters.

- **Stakeholder-Specific Interfaces:** The creation of dynamic, role-based visualization components. The dedicated “Student View” for filtering specific cohort schedules, the “Professor View,” and the macro-level “Master Occupancy” and “Free Room Finder” views ensure that every user receives actionable data without cognitive overload.
- **Modern Technical Architecture:** The deployment of a high-performance frontend utilizing React 19, TypeScript, and Vite, paired with a serverless Supabase backend. This stack ensures the application is highly responsive, strictly typed, and easily maintainable.
- **Professional Export Capabilities:** The integration of client-side PDF generation and automated Google Sheets API formatting, bridging the gap between dynamic web views and the necessity for static, offline documentation.

6.2 Current Limitations and Proposed Enhancements

No system is perfect, and ours is no exception. The limitations we have identified point directly to the most promising areas for future work. Table 6.1 summarises what we see as the key gaps and how they could be addressed.

6.3 Concluding Remarks

Looking back, we are proud of what we have built. The IIITA Timetabling System takes a problem that used to consume weeks of manual effort and solves it in seconds, right in the browser. Our (1+1)-ES solver reliably produces feasible, high-quality timetables; the drag-and-drop editor lets administrators fine-tune results with full constraint awareness; and

TABLE 6.1: System Limitations and Future Enhancements

Domain	Current Limitation	Proposed Enhancement
Data Ingestion	Heavily reliant on hardcoded regular expressions to parse II-ITA's specific Excel format.	Implementation of a dynamic mapping UI allowing users to visually map Excel columns to DB fields.
Solver Performance	Single-threaded (1+1)-ES running on the browser's main thread via batched <code>setTimeout</code> . Sufficient for current problem sizes but may become a bottleneck for larger institutions.	Migration to Web Workers for parallel solver execution, or a server-side solver for larger problem instances. Exploration of hybrid meta-heuristics (e.g., memetic algorithms combining ES with local search).
Solver Optimality	The Evolution Strategy is a meta-heuristic; solutions are feasible but not guaranteed to be globally optimal with respect to soft constraints.	Investigation of multi-objective optimization (Pareto-optimal fronts) and hybridization with constraint programming (CP) solvers for provable optimality on subproblems.
State Management	Relies on localized React state and custom hooks, potentially causing prop-drilling in deep components.	Adoption of a global state manager (e.g., Zustand or Redux) to optimize data sharing across views.
Notifications	Users must actively check the web portal for schedule updates.	Development of a Progressive Web App (PWA) with Push Notifications via Firebase Cloud Messaging.
Analytics	Basic occupancy visualization without long-term historical analysis.	Dedicated BI Dashboard to track room utilization metrics across multiple academic years.

the role-specific views ensure that every stakeholder—from first-year students to campus administrators—gets exactly the information they need. Perhaps most importantly, the project demonstrates that you do not need heavyweight server infrastructure to solve real scheduling problems: a well-designed, pure-TypeScript solver running on the client side is more than enough for a real-world university. We hope this work serves as a practical blueprint for other institutions looking to modernise their scheduling workflows.

Appendix A

Core Algorithm Snippets

This appendix provides crucial code excerpts from the React 19 codebase, demonstrating the practical implementation of the theoretical concepts discussed in the main chapters.

A.1 Dynamic Grid Generation Logic

The following TypeScript snippet from the `TimetableViewer` component demonstrates how the system extracts unique time boundaries to dynamically generate responsive column headers, preventing the display of empty, unused time blocks.

LISTING A.1: Dynamic Time Column Extraction Algorithm

```
// Extract all unique start and end times from the active slots
const allTimes = useMemo(() => {
  const times = new Set<string>();

  // Inject hardcoded institutional break times
  times.add("10:50");
  times.add("11:00");
  times.add("13:00");
  times.add("14:30");

  // Extract times from all valid timetable slots
  slots.forEach(slot => {
    if (slot.start_time && slot.end_time) {
      times.add(slot.start_time.substring(0, 5));
      times.add(slot.end_time.substring(0, 5));
    }
  });

  // Sort chronologically and format as standard columns
  return Array.from(times).sort().reduce((acc, curr, index, arr) => {
    if (index < arr.length - 1) {
      acc.push(`${curr}-${arr[index + 1]}`);
    }
    return acc;
  }, [] as string[]);
}, [slots]);
```

A.2 Real-Time Clash Detection Filter

This snippet illustrates the $O(N^2)$ comparison logic used to immediately identify Room, Professor, and Section overlaps within the currently selected semester block.

LISTING A.2: Overlap Detection and Categorization Logic

```
// O(N^2) comparison to find all overlapping events
const newClashes = filteredSlots.filter((slot, i) => {
  return filteredSlots.some((otherSlot, j) => {
    // Skip self-comparison
    if (i === j) return false;
    // Must be on the same day to clash
    if (slot.day_of_week !== otherSlot.day_of_week) return false;

    // Check for temporal intersection
    const isTimeOverlap = slot.start_time < otherSlot.end_time &&
      slot.end_time > otherSlot.start_time;

    if (isTimeOverlap) {
      // 1. Room Clash
      if (slot.rooms?.name === otherSlot.rooms?.name) return true;
      // 2. Professor Clash
      if (slot.professors?.name === otherSlot.professors?.name) return
        true;
      // 3. Section Clash (Same section, different subject)
      if (slot.student_groups?.name === otherSlot.student_groups?.name &&
        slot.subjects?.id !== otherSlot.subjects?.id) {
        return true;
      }
    }
  });
});
```

A.3 Solver Fitness Evaluation Function

The following excerpt from `constraints.ts` demonstrates the combined fitness evaluation that scores a candidate timetable against all ten hard constraints and three soft constraints.

LISTING A.3: Combined Fitness Evaluation (`constraints.ts`)

```
export function evaluate(input: SolverInput, solution: Solution):
  FitnessResult {
  const { sessions, rooms, numDays, config } = input;

  // Count all hard constraint violations
  const timeBoundary = countTimeBoundaryViolations(sessions, solution);
  const breakCrossing = countBreakViolations(sessions, solution);
  const roomOverlap = countRoomOverlaps(sessions, solution, rooms);
  const professorOverlap = countProfessorOverlaps(sessions, solution);
  const groupOverlap = countGroupOverlaps(sessions, solution);
  const electiveSync = countElectiveSyncViolations(sessions, solution);
  const labRoom = countLabRoomViolations(sessions, solution, rooms);
  const wmcSectionOverlap = countWMCSectionOverlaps(sessions, solution);
  const homeRoom = countHomeRoomViolations(sessions, solution);
  const twoOneLecture = countTwoOneLectureViolations(sessions, solution);

  ;

  const hardViolations = timeBoundary + breakCrossing + roomOverlap +
    professorOverlap + groupOverlap + electiveSync + labRoom +
    wmcSectionOverlap + homeRoom + twoOneLecture;
```

```

// Compute soft constraint penalties
const gapPenalty = computeGapPenalty(sessions, solution, numDays);
const roomUtilizationPenalty = computeRoomUtilizationPenalty(
  sessions, solution, rooms,
  config.roomUtilizationThreshold,
  config.roomOverutilizationThreshold,
);
const professorSameHalfPenalty = computeProfessorSameHalfPenalty(
  sessions, solution, numDays
);

// Weighted composite fitness
const total = hardViolations * config.hardPenalty
  + gapPenalty * config.gapWeight
  + roomUtilizationPenalty * config.roomUtilizationWeight
  + professorSameHalfPenalty * config.professorSameHalfWeight;

return { total, hardViolations, gapPenalty,
  roomUtilizationPenalty, professorSameHalfPenalty,
  violationBreakdown: { timeBoundary, breakCrossing,
    roomOverlap, professorOverlap, groupOverlap,
    electiveSync, labRoom, wmcSectionOverlap,
    homeRoom, twoOneLecture },
};
}

```

A.4 (1+1)-ES Solver Main Loop

The core optimization loop from `solver.ts`, showing elitist selection, σ -adaptation via the 1/5th success rule, and stagnation restart.

LISTING A.4: (1+1)-ES Main Loop (`solver.ts`)

```

export async function runSolver(
  input: SolverInput,
  onProgress?: (p: SolverProgress) => void,
  cancelToken?: { cancelled: boolean },
  seedSolution?: Solution,
  lockedGenes?: Gene[],
): Promise<SolverResult> {
  const { config } = input;
  let parent = seedSolution
    ? seedSolution.map(g => ({ ...g }))
    : generateInitialSolution(input);

  // Pin locked genes from published timetables
  if (lockedGenes) {
    const offset = input.sessions.length - lockedGenes.length;
    for (let i = 0; i < lockedGenes.length; i++) {
      parent[offset + i] = { ...lockedGenes[i] };
    }
  }

  let bestFitness = evaluate(input, parent);
  let bestSolution = parent.map(g => ({ ...g }));
  let sigma = config.initialSigma;

```

```

let successes = 0;

for (let gen = 1; gen <= config.maxGenerations; gen++) {
  if (cancelToken?.cancelled) break;

  const offspring = mutate(parent, input, sigma);
  // Re-pin locked genes (mutations must not move them)
  if (lockedGenes) { /* ... pin logic ... */ }
  const offFitness = evaluate(input, offspring);

  // Elitist selection: accept if offspring is <= parent
  if (offFitness.total <= bestFitness.total) {
    parent = offspring;
    bestFitness = offFitness;
    bestSolution = offspring.map(g => ({ ...g }));
    successes++;
  }

  // 1/5th rule sigma adaptation every W generations
  if (gen % config.adaptationWindow === 0) {
    const rate = successes / config.adaptationWindow;
    if (rate > 0.2) sigma *= config.sigmaIncrease;
    else if (rate < 0.2) sigma *= config.sigmaDecrease;
    successes = 0;
  }

  // Stagnation restart
  if (stagnationCounter > stagnationThreshold) {
    parent = generateInitialSolution(input);
    // Re-pin locked genes ...
  }

  // Early termination if feasible
  if (bestFitness.hardViolations === 0) break;
}
return { solution: bestSolution, fitness: bestFitness, ... };
}

```

A.5 LNS Auto-Fix for the Editor

The Large Neighbourhood Search routine from `localSearch.ts`, which identifies all clashing sessions and iteratively repairs them.

LISTING A.5: Full LNS Destroy-and-Repair (`localSearch.ts`)

```

export function runFullLNS(
  input: SolverInput,
  solution: Solution,
  maxAttempts: number = 1000,
): { solution: Solution; fitness: FitnessResult } | null {
  const baseFitness = evaluate(input, solution);
  if (baseFitness.hardViolations === 0) {
    return { solution, fitness: baseFitness };
  }

  // Find all session indices involved in violations
  let clashingIndices = findClashingIndices(input, solution);

```

```

let targets = Array.from(clashingIndices);
let bestSolution = solution.map(g => ({ ...g }));
let bestFitness = baseFitness;

for (let attempt = 0; attempt < maxAttempts; attempt++) {
  const candidate = bestSolution.map(g => ({ ...g }));
  // Pick a random clashing session
  const idx = targets[Math.floor(Math.random() * targets.length)];
  const session = input.sessions[idx];

  // Apply random mutation: relocate (50%), time shift (30%), room
  // (20%)
  const strategy = Math.random();
  if (strategy < 0.5)
    candidate[idx] = mutateRelocate(session, input.rooms);
  else if (strategy < 0.8)
    candidate[idx] = { ...candidate[idx],
      ...mutateTime(candidate[idx], session, 3.0) };
  else
    candidate[idx] = { ...candidate[idx],
      roomIndex: mutateRoom(session, input.rooms).roomIndex };

  const candidateFitness = evaluate(input, candidate);
  if (candidateFitness.total < bestFitness.total) {
    bestSolution = candidate;
    bestFitness = candidateFitness;
    if (bestFitness.hardViolations === 0) break;
    // Refresh targets after improvement
    clashingIndices = findClashingIndices(input, bestSolution);
    targets = Array.from(clashingIndices);
  }
}
return bestFitness.total < baseFitness.total
  ? { solution: bestSolution, fitness: bestFitness } : null;
}

```